

— WEB DEVELOPMENT

CSS Visual System: Controlling the Cascade

Master specificity, inheritance, and CSS variables to build maintainable, scalable design systems.

■ THE CASCADE

■ INHERITANCE

■ DESIGN TOKENS

The Cascade: Order of importance

When two CSS rules target the same element and property, the browser uses the Cascade to decide which rule wins.

01

Source Order

If two rules have the exact same weight, the one that appears **last** in the CSS file wins.

```
h1 { color: blue; }  
h1 { color: red; } /* Wins */
```

02

Specificity

A more specific selector will always override a less specific one, regardless of source order.

```
.title { color: red; } /* Wins */  
h1 { color: blue; }
```

03

Importance

The `!important` flag overrides normal specificity rules. Use it extremely sparingly, mostly for utility classes.

```
h1 { color: red !important; }  
#main .title { color: blue; }
```

Specificity: Calculating weight

Think of specificity as a four-part score. The browser compares scores from left to right to determine which rule wins.

1

INLINE

style="..." attribute

0

IDS

#header, #nav

0

CLASSES

.btn, :hover, [type]

0

ELEMENTS

h1, p, ::before

h1	0, 0, 0, 1
.card h1	0, 0, 1, 1
.card .title	0, 0, 2, 0
#hero .title	0, 1, 1, 0

Inheritance: Passing traits down

Some properties flow from parents to children, reducing repetition.

WHAT INHERITS?

Typography like color, font-family, and line-height inherit.
Set defaults on the body.

WHAT DOESN'T?

Layout props such as margin, padding, and width do not inherit.

```
body { color: #f47945; /* inherits */ }
```

`<body>` (color: #f47945)

`<article>` (no color set)

`<h2>`

Inherited Heading

`<p>`

Inherited paragraph text.

Custom Properties: CSS Variables

Stop repeating hex codes and font sizes. CSS variables let you store values in one place and reuse them throughout your stylesheet.

SINGLE SOURCE OF TRUTH

If your brand color changes, you only need to update it in one place. Every element using that variable updates automatically.

SEMANTIC NAMING

Instead of guessing what #f47945 means, you can name it --color-primary or --color-accent, making your code self-documenting.

1. DEFINE THE VARIABLE

```
:root { --color-primary: #f47945; }
```

2. USE THE VARIABLE

```
.button { background-color: var(--color-primary); }
```

Design Tokens: The single source of truth

By defining CSS variables on the `:root` pseudo-class, we create a global dictionary of design decisions called "tokens."

THE :ROOT SELECTOR

`:root` targets the highest-level element in the document (the `<html>` tag). Variables defined here are globally available to every element on the page.

SEMANTIC NAMING

Name variables by their **purpose**, not their value. Use `--color-primary` instead of `--purple`. If the brand color changes to blue, the variable name still makes sense.

```
/* Global Design Tokens */  
:root {  
  /* Colors */  
  --color-primary: #4d176e;  
  --color-accent: #f47945;  
  --color-bg: #fffdfb;  
  
  /* Typography */  
  --font-sans: 'Outfit', sans-serif;  
  --text-base: 1rem;  
}
```

Building a color system

Define your brand colors once in the root element. This ensures consistency and makes future updates or theme changes effortless.

```
/* Global Color Palette */

:root {
  /* Brand Colors */
  --color-primary: #4d176e;
  --color-accent: #f47945;

  /* UI Colors */
  --color-bg: #fffdfb;
  --color-text: #151116;
}
```

PRIMARY

--color-primary

ACCENT

--color-accent

BACKGROUND

--color-bg

TEXT

--color-text

Building a typography system

Establish a consistent typographic scale using CSS variables for font families, sizes, and weights.

```
:root {  
  /* Font Families */ --font-sans: 'Outfit', sans-serif;  
  --font-mono: 'Courier New', monospace;  
  /* Font Sizes (Modular Scale) */ --text-sm:  
0.875rem; --text-base: 1rem;  
  --text-lg: 1.25rem; --text-xl: 1.5rem;  
  --text-2xl: 2rem; --text-3xl: 3rem;  
  /* Weights */ --weight-normal: 400; --weight-  
bold: 700; }
```

`VAR(--TEXT-3XL) + VAR(--WEIGHT-BLACK)`

Display Heading

`VAR(--TEXT-XL) + VAR(--WEIGHT-BOLD)`

Section Title

`VAR(--TEXT-BASE) + VAR(--WEIGHT-NORMAL)`

Body text used for reading. Keep lines concise and readable.

`VAR(--TEXT-SM) + VAR(--WEIGHT-BOLD)`

METADATA & EYEBROWS

Building a spacing system

Replace arbitrary pixel values with a predictable scale. This ensures consistent margins, padding, and gaps across your entire interface.

```
/* Global Spacing Scale */
```

```
:root {
```

```
/* Base unit: 8px */
```

```
--space-xs: 4px;
```

```
--space-sm: 8px;
```

```
--space-md: 16px;
```

```
--space-lg: 32px;
```

```
--space-xl: 64px;
```

```
--space-2xl: 128px;
```

```
}
```

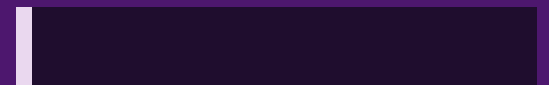
--space-xs

4px



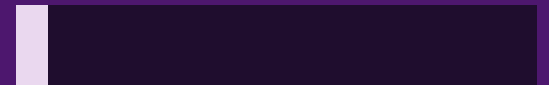
--space-sm

8px



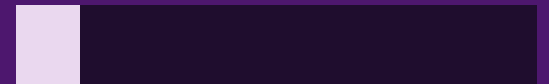
--space-md

16px



--space-lg

32px



--space-xl

64px



--space-2xl

128px



Applying the system

Replace hard-coded values with the `var()` function. This connects your components directly to the global design tokens.

■ BEFORE: HARD-CODED

```
.primary-button {  
  background-color: #4d176e;  
  color: #fffdfb;  
  font-size: 16px;  
  font-weight: 700;  
  padding: 16px 32px;  
}
```

■ AFTER: TOKENIZED

```
.primary-button {  
  background-color: var(--color-  
primary);  
  color: var(--color-bg);  
  font-size: var(--text-base);  
  font-weight: var(--weight-bold);  
  padding: var(--space-md) var(--space-  
lg);  
}
```

Theming and Dark Mode

Because CSS variables are dynamic, you can redefine them inside a media query or a class to instantly swap themes across your entire site.

```
/* Light */
:root { --bg-color: #fffdfb; --text-color:
#151116; --accent-color: #4d176e; }
/* Dark Override */
@media (prefers-color-scheme: dark) { :root { --
bg-color: #151116; --text-color: #ead8ef; --
accent-color: #f47945; } }
```

LIGHT MODE ACTIVE

Course Registration

Join the upcoming cohort to
master modern web
development techniques.

ENROLL NOW

DARK MODE ACTIVE

Course Registration

Join the upcoming cohort to
master modern web
development techniques.

ENROLL NOW

Lab: Implement design tokens

Refactor your programme page stylesheet. Replace all hard-coded values with a robust, maintainable CSS variable system.

DELIVERABLES

1. Define Tokens

Create a `:root` block defining variables for colors, typography, and spacing.

2. Refactor Colors

Replace all hex codes with your new `var(--color-...)` variables.

3. Refactor Type

Replace hard-coded font families and sizes with your typography tokens.

4. Refactor Space

Update margins and padding to use your modular spacing scale.

SUCCESS CRITERIA

- No hard-coded colors or font sizes remain in the CSS file.
- The visual appearance of the page remains exactly the same as before.
- Changing a single `var(--color-...)` variable in `:root` updates the entire page consistently.

EXTENSION CHALLENGE

Implement a dark mode theme. Use a `@media (prefers-color-scheme: dark)` query to redefine your color variables, automatically switching the theme based on system preferences.